

喜马拉雅小程序插件 v2.1

更新日期：2026-07-02

一、概述

本插件由喜马拉雅提供，合作方将其集成到**宿主小程序**后，即可在小程序内嵌入喜马拉雅儿童内容（小喜马）的完整浏览与播放体验，包括：首页、搜索、专辑详情、分类、个人中心等页面。

插件与宿主小程序之间通过微信原生的 **EventChannel** 进行双向通信，插件同时通过 `requirePlugin()` 向宿主暴露若干查询接口和协议常量，宿主可通过 `plugin-exports.js` 向插件提供数据接口。

二、接入前准备

事项	说明
获取插件 AppID	联系喜马拉雅侧分配
获取合作方 AppKey	喜马拉雅分配，用于标识合作方身份
微信开放平台权限	确认宿主小程序已开启"使用插件"能力
设备 ID (deviceId)	由合作方生成/维护，标识终端设备
RSA 加密公钥（静默登陆）	喜马拉雅提供，用于加密用户手机号生成静默登陆 ticket（可选，仅需静默登陆时申请）

合作方小程序申请喜马儿童SDK插件权限

路径：微信公众平台 -> 登陆小程序 -> 账号设置 -> 第三方设置里'插件管理'搜索添加插件

- 添加喜马儿童SDK插件权限示例
- 完成添加后通知喜马人员通过申请（申请只有24h有效）

合作方小程序申请喜马小程序拉起半屏小程序的权限

- 喜马拉雅提供小程序 AppID
- 合作方在半屏小程序管理申请权限；路径：微信公众平台 -> 登陆小程序 -> 账号设置 '第三方设置' -> 半屏小程序管理（插件管理下方） -> 申请喜马拉雅小程序权限
- 完成添加后通知喜马拉雅人员通过申请（申请只有24h有效）

三、插件注册

在宿主小程序的 `app.json` 中添加以下声明：

```
{
  "plugins": {
    "xmlyPlugin": {
      "version": "1.0.0",
      "provider": "<插件AppID>",
      "export": "plugin-exports.js"
    }
  },

  "embeddedAppIdList": [<喜马拉雅小程序AppID>]
}
```

复制代码

说明：“export” 字段用于声明宿主向插件暴露的接口文件。插件会通过 `requireMiniProgram()` 调用该文件中导出的方法（如 VIP 票据、设备 ID 等），详见 [第七节](#)。

四、启动插件页面

4.1 对外入口页面

宿主小程序可直接跳转的页面共 4 个。`albumId` / `trackId` 由喜马拉雅提供，如需直跳特定内容请联系喜马拉雅获取。所有路径均需携带 `appKey` 参数（见第九节）。

页面	参数名	类型	必填	参数说明	路径示例
首页	—	—	—	—	<code>plugin://xmlyPlugin/home?albumId=YOUR_APP_KEY</code>


```
success: function(res) {  
  // 保存 channel 引用，用于向插件发送消息  
  this.eventChannel = res.eventChannel;  
}  
});
```

注意：若不传 `events`，EventChannel 将无法建立，播放通知与登录通知均不会到达宿主。

五、EventChannel 消息协议

插件与宿主之间的实时消息通过 EventChannel 双向流动。所有事件名和消息类型枚举均通过 `requirePlugin()` 从插件侧统一导出，**宿主不应硬编码任何字符串**：

∨

复制代码

```
const plugin = requirePlugin('xmlyPlugin');
```

// EventChannel 事件名

plugin.ChannelEvents.PLUGIN_TO_HOST // 宿主监听插件消息时注册的事件名
plugin.ChannelEvents.HOST_TO_PLUGIN // 宿主向插件发消息时使用的事件名

// 插件 → 宿主 消息类型枚举

plugin.PluginMessageTypes.PLAY // 播放通知
plugin.PluginMessageTypes.LOGIN_COMPLETE // 登录完成通知

// 宿主 → 插件 消息类型枚举

plugin.HostMessageTypes.NEED_PAYMENT // 通知插件拉起支付

方向	事件名常量	说明
插件 → 宿主	<code>plugin.ChannelEvents.PLUGIN_TO_HOST</code>	插件主动推送（播放、登录等）
宿主 → 插件	<code>plugin.ChannelEvents.HOST_TO_PLUGIN</code>	宿主主动通知（支付等）

5.1 插件 → 宿主消息

消息信封结构

```
interface IPluginMessage<T> {  
    type: string;           // 消息类型，见 plugin.PluginMessageTypes  
    version: string;        // 协议版本，当前固定 "1.0"  
    timestamp: number;      // 发送时间戳 (ms)  
    data: T;                // 业务载荷，结构因 type 而异  
}
```

复制代码

play — 播放通知

用户在插件内触发播放时，插件通知宿主进行实际的音频播放。

```
// data 结构  
interface IPlayMessageData {  
    albumId: string,        // 专辑 ID  
    trackId: string,        // 单集 ID  
    breakSecond: number,    // 断点续播秒数 (首次为 0)  
    duration?: number,      // 音频时长 (秒)  
    albumTitle?: string,    // 专辑名称  
    trackTitle?: string,    // 单集名称  
}
```

复制代码

宿主处理示例：

```
case plugin.PluginMessageTypes.PLAY:  
    const { albumId, trackId, breakSecond } = message.data;  
    myPlayer.play({ albumId, trackId, startSecond: breakSecond });  
    break;
```

复制代码

login_complete — 登录完成通知

用户在插件内完成登录后，插件将 token 信息通知宿主，便于宿主同步登录态。

```
// data 结构  
interface ILoginCompleteData {
```

复制代码

```
access_token: string;    // 喜马拉雅 access token
uid?: string | number;  // 用户 ID
expires_in?: number;    // token 有效期（秒）
}
```

宿主处理示例：



复制代码

```
case plugin.PluginMessageTypes.LOGIN_COMPLETE:
  const { access_token, uid, expires_in } = message.data;
  storage.set('xmly_token', access_token);
  break;
```

消息处理完整示例



复制代码

```
const plugin = requirePlugin('xmlyPlugin');

function handlePluginMessage(message) {
  switch (message.type) {
    case plugin.PluginMessageTypes.PLAY:
      myPlayer.play({
        albumId: message.data.albumId,
        trackId: message.data.trackId,
        startSecond: message.data.breakSecond,
      });
      break;
    case plugin.PluginMessageTypes.LOGIN_COMPLETE:
      storage.set('xmly_token', message.data.access_token);
      break;
    default:
      console.warn('[xmlyPlugin] 未知消息类型:', message.type);
  }
}
```

5.2 宿主 → 插件消息

宿主保存 `eventChannel` 引用后，可主动向插件发送消息：



复制代码

```
interface IHostMessage<T> {
  type: string; // 消息类型，见 plugin.HostMessageTypes
  version: string; // 协议版本，当前固定 "1.0"
  timestamp: number; // 发送时间戳 (ms)
  data?: T; // 业务载荷 (可选)
}
```

need_payment — 通知插件拉起支付

当硬件设备播放到付费内容时，宿主通知插件弹出购买引导。

▽ 复制代码

```
// data 结构 (可选)
interface INeedPaymentData {
  title?: string; // 弹窗提示文案，不传时插件使用默认文案
}
```

宿主发送示例：

▽ 复制代码

```
const plugin = requirePlugin('xmlyPlugin');

// 触发场景示例：设备播放到付费内容时调用
function notifyNeedPayment(customTitle) {
  if (!this.eventChannel) {
    console.warn('[host] eventChannel 未建立，请先通过 navigateTo 进入插件');
    return;
  }
  this.eventChannel.emit(plugin.ChannelEvents.HOST_TO_PLUGIN, {
    type: plugin.HostMessageTypes.NEED_PAYMENT,
    version: '1.0',
    timestamp: Date.now(),
    data: customTitle ? { title: customTitle } : {},
  });
}
```

六、插件导出接口 (requirePlugin)

通信方向：宿主小程序（合作方）→ 小程序插件

宿主主动调用插件暴露的接口，同步获取插件内部状态或协议常量，无需等待事件推送。

宿主可通过 `requirePlugin()` 主动查询插件内部状态：



复制代码

```
const plugin = requirePlugin('xmlyPlugin');
```

6.1 `getTokenInfo()` — 获取 token 信息



复制代码

```
const tokenInfo = plugin.getTokenInfo();  
// 返回值：  
// {  
//   access_token: string | null,  
//   refresh_token?: string,  
//   uid?: string | number,  
//   expires_in?: number,  
//   scope?: string  
// }
```

6.2 `getPlatformInfo()` — 获取平台信息



复制代码

```
const platformInfo = plugin.getPlatformInfo();  
// 返回值: { platform: 'plugin', version: '1.0.0' }
```

6.3 协议常量

所有 EventChannel 事件名和消息类型枚举均通过插件统一导出（见 [第五节](#)），宿主侧应使用这些常量替代硬编码字符串。

6.4 `getUserInfo()` — 获取当前登陆用户信息



复制代码

```
// 获取当前登录用户信息（未登录时返回 null）  
const userInfo = plugin.getUserInfo();  
// { id, nickname, avatar_url, is_vip, vip_expired_at, ... }  
  
// 返回类型
```



```
interface UserInfo {
  id: number;
  nickname: string;
  avatar_url: string;
  is_verified: boolean;
  // 是否vip
  is_vip: boolean;
  // vip到期时间
  vip_expired_at: number;
}
```

七、宿主导出接口（plugin-exports.js）

通信方向：小程序插件 → 宿主小程序（合作方）

插件通过 `requireMiniProgram()` 主动拉取宿主提供的数据，如 VIP 票据、设备 ID 等。

宿主需在项目根目录创建 `plugin-exports.js`，并在 `app.json` 的插件声明中通过 `"export"` 字段指定（见第三节）。

7.1 接口清单

方法名	返回值	说明
<code>getVipTicket()</code>	<code>{ ticket: string }</code>	返回 VIP 权益票据
<code>getVipDeviceInfo()</code>	<code>{ device_id: string }</code>	返回第三方设备唯一标识
<code>getAuthTicket()</code>	<code>{ ticket: string }</code>	返回静默登陆 ticket（如果做静默登陆必须实现）

7.2 实现模版（此处只是示例）

```
// plugin-exports.js
module.exports = {
  /**
   * 返回 VIP 权益票据
   * @returns {{ ticket: string }}
   */
  getVipTicket() {
```

复制代码

```

// 替换为实际获取 ticket 的逻辑
return {
    ticket: 'YOUR_VIP_TICKET',
};
},

/**
 * 返回第三方设备 ID
 * @returns {{ device_id: string }}
 */
getVipDeviceInfo() {
    // 替换为实际的设备 ID
    return {
        device_id: 'YOUR_DEVICE_ID',
    };
},

/**
 * 返回RSA加密后的手机号
 * @returns {{ device_id: string }}
 */
getAuthTicket() {
    // 替换成实际的手机号加密流程，示例加密代码可参考7.3章节
    return {
        device_id: 'YOUR_AUTH_TICKET',
    };
},
};

```

说明：插件未能获取到宿主数据时会降级处理，不影响核心播放流程，但 VIP 相关功能可能受限。

7.3 静默登陆（getAuthTicket）

前置准备：

资源	来源
用户手机号	合作方自有
RSA 加密公钥	喜马拉雅提供

用户已同意自动登陆协议

合作方在自有页面完成授权确认，用户未同意时禁止调用静默登陆流程

ticket 生成流程：



复制代码

用户手机号

- RSA 公钥加密（PKCS#1 v1.5 或 OAEP，以喜马拉雅提供的公钥为准）
- 密文转 Hex 编码
- 即为 ticket

手机号加密示例（Node.js）：



复制代码

```
const crypto = require('crypto');

/**
 * 生成静默登陆 ticket
 * @param {string} phone 用户手机号（明文）
 * @param {string} publicKey 喜马拉雅提供的 RSA 公钥（PEM 格式）
 * @returns {string} Hex 编码的密文，直接作为 ticket 使用
 */
function generateAuthTicket(phone, publicKey) {
  const encrypted = crypto.publicEncrypt(
    {
      key: publicKey,
      padding: crypto.constants.RSA_PKCS1_PADDING,
    },
    Buffer.from(phone, 'utf8')
  );
  return encrypted.toString('hex');
}

// plugin-exports.js 中调用（ticket 应由服务端下发，不要在小程序端持有公钥）
module.exports = {
  async getAuthTicket() {
    const ticket = await fetchTicketFromYourServer(); // 向自有服务端请求
    return { ticket };
  },
};
```

八、支付能力说明

插件内购买会员时，支付逻辑由插件侧处理，无需宿主额外实现支付：

平台	行为
Android	插件自动跳转喜马拉雅支付小程序（ <code>wx.openEmbeddedMiniProgram</code> ）
iOS（Apple 政策限制）	插件从喜马拉雅服务端获取支付二维码图片，展示给用户扫码

宿主无需做任何支付相关处理。若宿主侧硬件设备播放到付费内容需引导购买，可通过第五节第 5.2 小节的 `need_payment` 消息通知插件弹出购买引导。

九、连接参数（AppKey / DeviceId）

插件启动时通过 URL 参数获取合作方标识，`**appKey` 为必传**，缺少时插件将使用内置默认值，导致数据归属异常。

在跳转插件任意页面时，URL 中携带以下参数：

参数名	类型	必填	说明
<code>appKey</code>	string	是	喜马拉雅分配的合作方标识，用于数据归因和权限校验，不传将使用内置默认值导致数据归属异常
<code>deviceId</code>	string	否	终端设备唯一标识；传入则使用传入值，不传则插件内部自动生成 UUID 并持久化存储，后续复用



复制代码

```
const plugin = requirePlugin('xmlyPlugin');

wx.navigateTo({
  url: 'plugin://xmlyPlugin/home?
appKey=YOUR_APP_KEY&deviceId=YOUR_DEVICE_ID',
  events: {
    [plugin.ChannelEvents.PLUGIN_TO_HOST]: function(message) {
      handlePluginMessage(message);
    }
  },
  success: function(res) {
    this.eventChannel = res.eventChannel;
  }
});
```

十、接入检查清单

- ☐ 插件权限及半屏小程序权限申请 以及一些appKey、RSA公钥申请（见第二节）
- ☐ 实现
- ☐ 使用
- ☐ 保存
- ☐ 根据业务需要实现 暴露给plugin的方法
- ☐ EventChannel 通信通道建立，及合作方内部完成相应业务处理
- ☐ 验证 iOS / Android 双端效果

十一、联系与支持

如有接入问题，请联系喜马拉雅对接负责人，并提供：

1. 宿主小程序 AppID
2. 复现步骤及截图/日志
3. `message.type`（如涉及消息通信问题）

十二、参考资料

- [微信小程序插件使用文档](#)
- [微信半屏小程序文档](#)